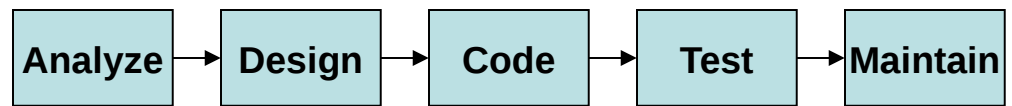
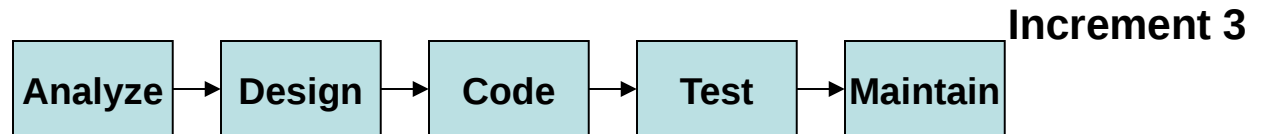
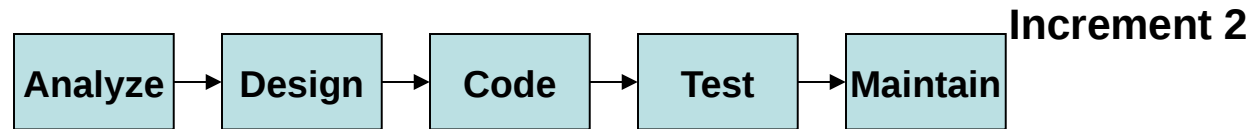
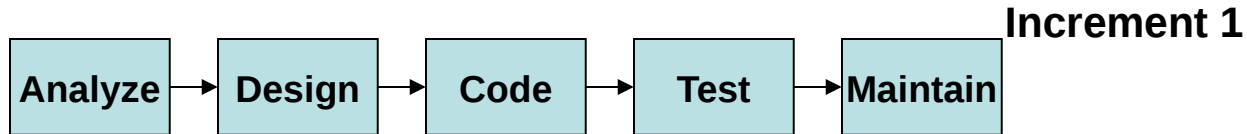


Evolutionary software process models

- Much of software is evolutionary in nature and is developed over many years with several incremental functionalities.
- Two popular models exist for evolutionary software:
 - Incremental model
 - Spiral model

Incremental model



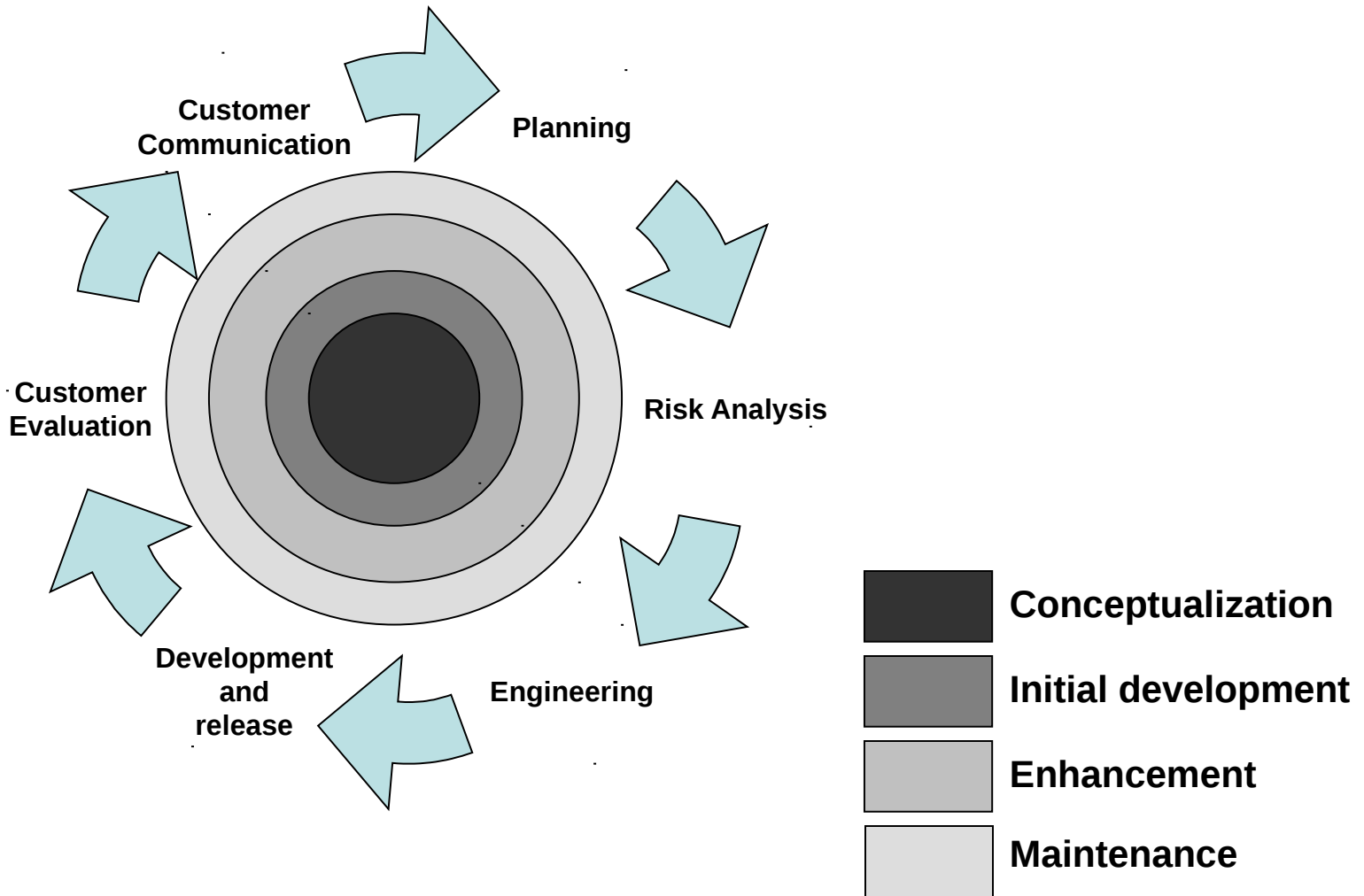
Time



Incremental model

- Delivers incremental versions of the product.
- Typically like the prototype model, with the style of waterfall development in each increment.
 - Unlike a prototype, this model delivers a working version with every increment.
- Very useful when produce requirements are evolving and team size is less.

Spiral model



Spiral model

- Here again, software is developed in incremental stages.
- First pass through the spiral will result in defining the product specification, second pass will be development of prototype, third pass will be system design followed by development and subsequent passes for refinement and maintenance.

Concurrent development model

- Very useful for development of large scale systems.
- Need very good clarity during planning to identify tasks or phases that will be concurrently done.
- Tasks within a concurrent entity are typically serial in nature.
- Safety critical systems standards sometimes insist on concurrent development resulting in two copies of the same system.

Component based development

- A set of pre-built, standardized software components are made available to put together in a specified fashion and create software.
- This process encompasses two parallel activities:
 - Domain engineering
 - Component based development

Component based development

- **Domain Engineering:** Typically explores an application domain with the specific intent of finding functional, behavioral and data components that can be re-used.
- **Component based development:** Collects requirements, selects appropriate architectural style to meet the objectives and
 - selects components for reuse,
 - reviews and modifies the components to ensure that they properly fit into the architecture of the system and
 - integrates the components to form sub-systems and finally, the full system.

Model based development

- Focuses mainly on requirements and design phases of development.
- Formal models are developed to capture requirements and design.
- Models are extensively validated to check for ambiguities, inconsistencies, feasibility, re-usability etc.
- Typically, code, test cases and documentation are automatically generated.

Agile methods for development

- Values central to agile methodologies are:
 - **Individuals and interactions** over processes and tools
 - **Working software** over comprehensive documentation
 - **Customer collaboration** over contract negotiation
 - **Responding to change** over following a plan

Characteristics of agile processes

- Modularity on development process level
- Iterative fast cycles enabling fast verifications and corrections
- Time-bound with iteration cycles from one to six weeks
- Parsimony in development process, removing all unnecessary activities
- Adaptive with possible emergent new risks
- Incremental process approach that allows application building in small steps
- People oriented
- Collaborative and communicative working style

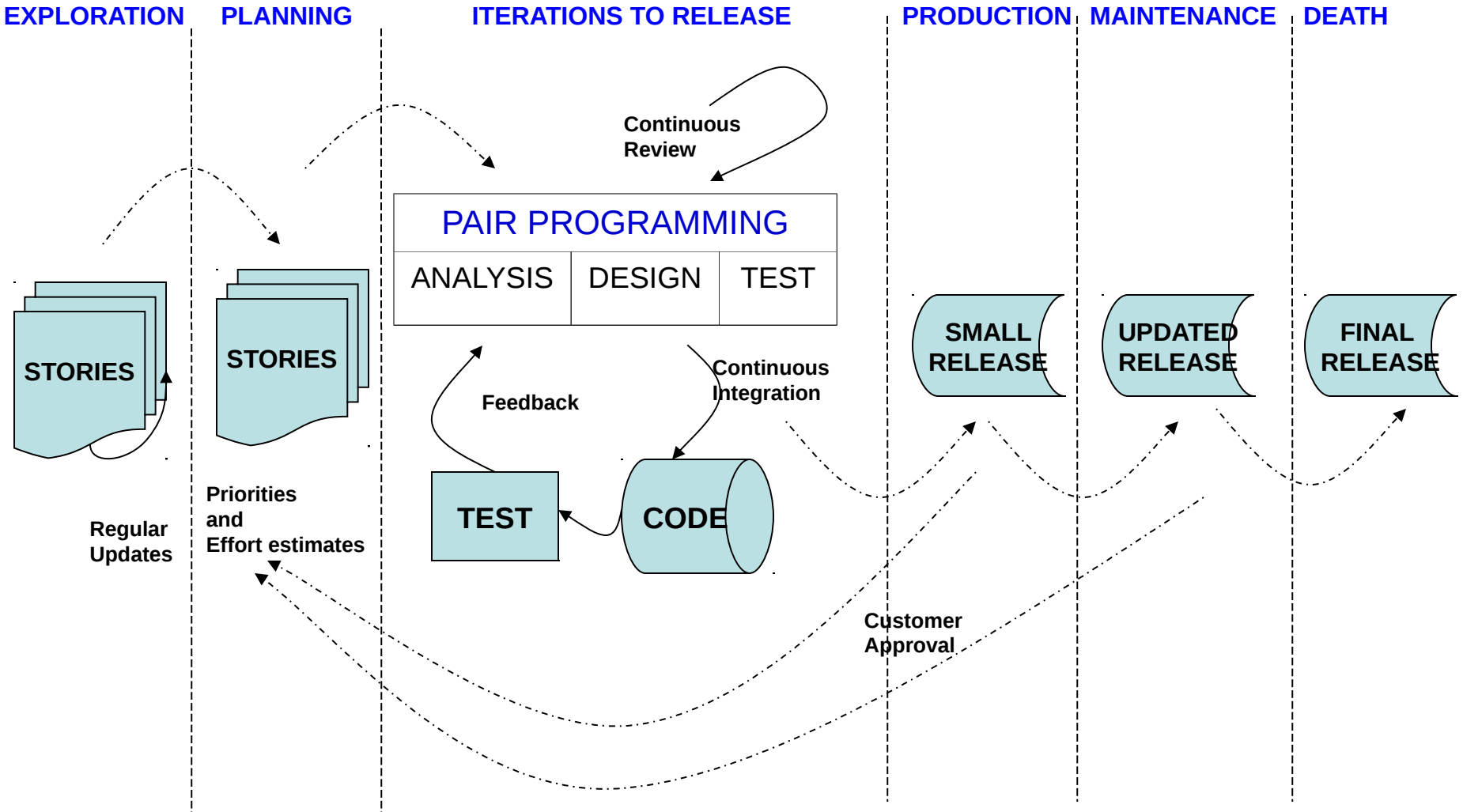
Agile methods: Goals

- Agile methods are designed to
 - Produce first delivery within weeks: achieves early win and quick feedback
 - Invent simple solutions: there is less to change and changes are easier
 - Improve design quality continually: next iteration is less expensive to implement
 - Test constantly: Earlier, less expensive defect detection and correction

Some existing agile methods

- Extreme Programming (XP)
- Scrum
- Crystal methodologies
- Feature driven development
- Rational Unified Process (RUP)
- Adaptive software development

Extreme Programming (XP)



XP: Phases

- **Exploration phase:**
 - Customers write out story cards describing the required features.
 - Teams familiarize themselves with the technologies, tools and practices.
 - Lasts between a few weeks to a few months based on the project.
- **Planning phase:**
 - Stories are prioritized, contents for first small release are decided and effort estimation and initial scheduling are done.
 - Lasts for a couple of days.

XP: Phases

- Iterations to release phase:
 - Includes several iterations before the first release: customer decided stories to be implemented in each iteration
 - Schedule is broken into a number of iterations, each taking one to four weeks for implementation.
 - Functional tests are run jointly with the customer at the end of every iteration.
 - First iteration typically results in architecture for the whole system and the last iteration results in production.

XP: Phases

- **Productionizing phase:**
 - Requires extra testing before release to customer.
 - Iterations are shorter than earlier phase.
 - Changes can still come, sometimes deferred to be implemented in the next phase.
- **Maintenance phase:**
 - Keeps the system running while catering to new iterations.
- **Death phase:**
 - Customer no longer has stories to implement.
 - Documentation is finally written.

XP: Roles and responsibilities

- Programmer
- Customer
- Tester
- Tracker
- Coach
- Consultant
- Manager